

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

NIVEL 3 (AVANZADO)

Curso 3: Creación y Desarrollo de Agentes de IA

«Modelos LLM, Inferencia, Tool Calling, Memoria Vectorial, RAG y Arquitecturas ReAct»

Manual completo del curso compilado oficialmente. Incluye fundamentación teórica, metáforas conceptuales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor Principal: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Total de Módulos / Clases: 3 | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura del Manual de Curso

Este libro unificado contiene el desglose secuencial de cada módulo formativo, estructurado con rigor de ingeniería y diseñado para servir como manual de referencia permanente.

Plan de Estudios y Hoja de Ruta

El curso está dividido en las siguientes unidades temáticas secuenciales:

Código	Unidad y Metáfora	Objetivo Principal
Módulo 01	Módulo 01: Fundamentos de IA Generativa y LLMs «El Cerebro Probabilístico y el Molde de Salida»	Comprender la naturaleza probabilística de los LLMs, el cálculo de tokens, la ve...
Módulo 02	Módulo 02: Herramientas, Memoria y RAG «La Caja de Herramientas y el Bibliotecario con Memoria»	Comprender cómo un modelo utiliza herramientas externas para ejecutar código y c...
Módulo 03	Módulo 03: Construcción de Agentes Autónomos «El Ciclo Cognitivo ReAct y el Enjambre de Agentes»	Comprender la diferencia entre un script lineal y un bucle de razonamiento autón...

Criterio de Progresión

Cada unidad construye sobre los cimientos de la anterior. Se recomienda dominar la teoría, analizar el diagrama de arquitectura y replicar el código de cada sección antes de avanzar a la siguiente.

Módulo 01: Fundamentos de IA Generativa y LLMs

Los LLMs no 'piensan' como los humanos; son gigantescas redes neuronales que predicen la siguiente palabra más probable dado un contexto.

☀ **Metáfora Central: El Cerebro Probabilístico y el Molde de Salida**

Un LLM es como un erudito que ha leído toda la biblioteca de Alejandría; si le haces una pregunta abierta responderá con fluidez literaria, pero si le colocas un molde rígido (un esquema JSON con Pydantic), vertirá su conocimiento exclusivamente dentro de la forma exacta que necesitas.

Principios Teóricos y Modelo Mental

Tokens y Contexto: Los textos se tokenizan en fragmentos sub-palabra; la ventana de contexto limita cuántos tokens puede procesar simultáneamente.

Parámetros Clave: Temperatura (0.0 para respuestas deterministas y código; 0.7+ para creatividad), Top-P y penalización de repetición.

⚡ **Regla de Oro en Python**

En entornos de producción nunca uses texto libre del LLM; fuerza siempre salidas tipadas estructuradas validadas con Pydantic.

Pipeline de Inferencia y Validación Estructurada

Flujo desde la construcción del System Prompt hasta la validación del objeto de salida.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Construcción del System Prompt con instrucciones de rol y Few-Shot examples.	Tokenización del prompt
2. Evaluación	Envío a la API del modelo (Gemini / OpenAI / Ollama) con schema JSON.	Inferencia en la GPU
3. Transformación	El modelo genera un payload JSON estricto cumpliendo la especificación.	Payload JSON crudo
4. Retorno / Salida	Pydantic parsea y valida los tipos de datos en un objeto Python listo.	Instancia BaseModel validada

Visualización Mental

Trata al LLM como un microservicio no determinista: coloca siempre una capa de validación antes de entregar los datos a tu backend.

Extractor de Entidades con Pydantic y LLM

main.py (Python 3.10+)

PEP 8 Compliant

```
from pydantic import BaseModel, Field
from typing import List

# Esquema de validación estricta
class AnalisisSentimiento(BaseModel):
    sentimiento: str = Field(description="POSITIVO, NEGATIVO o NEUTRO")
    puntuacion_confianza: float = Field(ge=0.0, le=1.0)
    temas_clave: List[str] = Field(default_factory=list)
    resumen_ejecutivo: str

# Simulación de respuesta parseada por el motor
payload_llm = '''{
  "sentimiento": "POSITIVO",
  "puntuacion_confianza": 0.96,
  "temas_clave": ["soporte rápido", "calidad software", "precio justo"],
  "resumen_ejecutivo": "El cliente expresa gran satisfacción con la atención recibida."
}'''

analisis = AnalisisSentimiento.model_validate_json(payload_llm)
print(f"Sentimiento: {analisis.sentimiento} ({analisis.puntuacion_confianza*100:.1f}%)")
print(f"Temas: {'', '.join(analisis.temas_clave)}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Confiar ciegamente en que el LLM siempre responderá JSON válido sin capturar excepciones de parseo o alucinaciones.

✗ Antipatrón

```
data = json.loads(llm_response) # Fallará si el LLM
incluye texto extra
```

✓ Patrón Correcto

```
try:
    data = Model.model_validate_json(llm_response)
except ValidationError as e:
    # Estrategia de reintento / corrección
```

Módulo 02: Herramientas, Memoria y RAG

Un LLM aislado solo puede generar texto; cuando le otorgas herramientas y memoria, se transforma en un agente inteligente capaz de interactuar con el mundo.

☀ **Metáfora Central: La Caja de Herramientas y el Bibliotecario con Memoria**

El LLM es como un consultor brillante pero encerrado en una habitación: Tool Calling es darle un teléfono para llamar a APIs o consultar bases de datos. RAG es ponerle al lado a un bibliotecario que busca en segundos los documentos exactos de tu empresa y se los pasa antes de que responda.

Principios Teóricos y Modelo Mental

Tool Calling: El LLM decide qué función invocar y genera los argumentos exactos; tu código ejecuta la función y devuelve el resultado al LLM.

Embeddings Vectoriales: Representaciones numéricas de significado semántico; textos con significados similares tienen alta similitud de coseno en el espacio vectorial.

⚡ **Regla de Oro en Python**

RAG resuelve el problema del conocimiento desactualizado y las alucinaciones sin necesidad de reentrenar el modelo fundacional.

Arquitectura de un Sistema RAG (Retrieval-Augmented Generation)

Flujo de ingestión, búsqueda semántica y generación contextualizada con documentos privados.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Interpreté	Estado en Memoria
1. Inicialización	Los documentos se dividen en fragmentos (chunks) y se vectorizan con un modelo de embeddings.	Vectores en Vector DB
2. Evaluación	El usuario hace una pregunta; se vectoriza la consulta del usuario.	Query vector
3. Transformación	Se realiza búsqueda de similitud (k-Nearest Neighbors / Cosine Similarity) para extraer los chunks más relevantes.	Contexto recuperado
4. Retorno / Salida	Se ensambla el prompt enriquecido [Contexto + Pregunta] y el LLM formula la respuesta final respaldada en hechos.	Respuesta precisa y sin alucinación

Visualización Mental

RAG es el examen a libro abierto del LLM: en lugar de memorizar todo, le das el párrafo exacto donde está la respuesta.

Definición y Ejecución de Herramientas (Tool Calling)

main.py (Python 3.10+)

PEP 8 Compliant

```
import json

# 1. Definición de la Herramienta en Python puro
def consultar_clima(ciudad: str) -> str:
    """Consulta la temperatura actual de una ciudad."""
    datos = {"Madrid": "24°C Despejado", "Bogotá": "18°C Lluvioso"}
    return datos.get(ciudad, "20°C Clima templado")

# 2. Registro de herramientas disponibles para el agente
HERRAMIENTAS_DISPONIBLES = {"consultar_clima": consultar_clima}

# 3. Simulación de orden de Tool Call emitida por el LLM
llamada_modelo = {
    "funcion": "consultar_clima",
    "argumentos": {"ciudad": "Madrid"}
}

# 4. Despachador de ejecución segura
nombre_fn = llamada_modelo["funcion"]
args = llamada_modelo["argumentos"]
resultado_tool = HERRAMIENTAS_DISPONIBLES[nombre_fn](**args)

print(f"Resultado de la herramienta para el LLM: {resultado_tool}")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Fragmentar documentos en chunks demasiado grandes (que diluyen la relevancia semántica) o demasiado pequeños (que pierden contexto).

❌ Antipatrón

Chunks de 5000 tokens: el embedding pierde especificidad semántica

✅ Patrón Correcto

Chunks de 300-500 tokens con 50 tokens de solapamiento (overlap)

Módulo 03: Construcción de Agentes Autónomos

Un agente autónomo no ejecuta un camino fijo; observa su entorno, razona sobre el objetivo, decide qué herramienta usar y evalúa los resultados de forma iterativa.

☀️ Metáfora Central: El Ciclo Cognitivo ReAct y el Enjambre de Agentes

El ciclo ReAct es como un detective privado resolviendo un misterio: tiene un Pensamiento (Thought: 'Necesito ver la cámara de seguridad'), realiza una Acción (Action: busca el video con una herramienta), analiza la Observación (Observation: 'El sospechoso salió a las 10:00'), y repite el ciclo hasta llegar a la Respuesta Final.

Principios Teóricos y Modelo Mental

Ciclo Cognitivo: Thought (Razonamiento interno) -> Action (Invocación de herramienta) -> Observation (Lectura del entorno) -> Evaluación.

Sistemas Multi-Agente: División de trabajo entre agentes especializados (Investigador, Programador, Auditor de Calidad) orquestados por un Supervisor.

⚡ Regla de Oro en Python

Todo bucle de agente autónomo debe tener un límite estricto de pasos máximos (`max_iterations`) para evitar bucles infinitos y consumo desmedido de tokens.

Grafo Cíclico de Razonamiento ReAct

Flujo de control dinámico donde el agente decide autónomamente continuar investigando o emitir la solución final.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Recibe la misión del usuario y formula el primer pensamiento estratégico.	Thought 1 formulado
2. Evaluación	Emite una orden de acción hacia una herramienta específica.	Action: Tool invocation
3. Transformación	Recibe la observación del entorno con los datos reales generados.	Observation incorporada al prompt
4. Retorno / Salida	¿Objetivo cumplido? Si no, repite ciclo; si sí, genera Final Answer.	Solución entregada



Visualización Mental

El agente mantiene un historial acumulativo de pensamientos y observaciones pasadas para no repetir errores.

Motor de Agente ReAct en Python Puro

main.py (Python 3.10+)

PEP 8 Compliant

```
class AgenteReAct:
    def __init__(self, herramientas: dict, max_pasos: int = 5):
        self.herramientas = herramientas
        self.max_pasos = max_pasos
        self.memoria: list[str] = []

    def ejecutar_mision(self, objetivo: str) -> str:
        self.memoria.append(f"Objetivo: {objetivo}")
        for paso in range(1, self.max_pasos + 1):
            print(f"\n--- [Paso {paso}] Ciclo Cognitivo ---")
            # 1. Simulación de pensamiento y decisión del LLM
            pensamiento = "Consultar base de datos para extraer métricas"
            accion_tool = "consultar_db"

            print(f"🧠 Thought: {pensamiento}")
            print(f"🔧 Action: {accion_tool}()")

            # 2. Ejecución de la herramienta y observación
            observacion = "Ventas del mes: $45,000 USD (Crecimiento +12%)"
            print(f"👁️ Observation: {observacion}")

            # 3. Condición de término
            return f"Respuesta Final: Las ventas crecieron un 12% alcanzando $45,000 USD."
        return "Límite de pasos alcanzado."
```

⚠️ Gotcha Frecuente (Trampa de Principiante)

Permitir que un agente ejecute comandos en el sistema operativo o mutaciones destructivas en bases de datos sin una capa de confirmación humana (Human-in-the-loop).

❌ Antipatrón

Agente ejecutando `rm -rf` o `DROP TABLE` sin validación

✅ Patrón Correcto

Validar permisos y requerir confirmación antes de acciones críticas

Conclusiones Generales del Curso

Has completado el recorrido integral de **Curso 3: Creación y Desarrollo de Agentes de IA**. El dominio de estos conceptos te sitúa con una ventaja competitiva sólida para el desarrollo de software profesional e Inteligencia Artificial.

Hitos Alcanzados

Comprensión profunda de la arquitectura, asimilación de patrones de diseño idiomáticos en Python, resolución de problemas algorítmicos y capacidad de integración de proyectos reales.

Fuentes Bibliográficas Canónicas

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica del lenguaje	docs.python.org/3/
PEP 8 - Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones profesionales de ingeniería	realpython.com
Suite wisrovi en GitHub	Librerías open source de alto rendimiento	github.com/wisrovi

Agradecimiento del Instructor

Gracias por tu disciplina, curiosidad y compromiso en cada lección. Continúa pedaleando y construyendo software con propósito. ¡Nos vemos en el siguiente nivel!